

Computer Vision 2: Exercise Sheet 1

Summary:

1. Review: Python and NumPy
2. Getting started with PyTorch
3. Using PyTorch as an automatic differentiation engine

1 Python and NumPy

You should be familiar with the Python programming language and the NumPy package for scientific computing. It is especially important that you understand well the concept of **multidimensional arrays**, how to manipulate them (indexing, reshaping, modifying dimensions), and how mathematical operations on arrays work. It may also be useful for you to occasionally use the Matplotlib library for plotting data.

If you do not feel confident in these topics yet, review the tutorial by Justin Johnson at cs231n.github.io/python-numpy-tutorial/.

2 Getting started with PyTorch

PyTorch is a Python-based scientific computing package that may be used as a replacement for NumPy to use the power of graphics processing units (GPUs), or as a fast and flexible deep learning research platform.

2.1 Install PyTorch

Follow the instructions found at <https://pytorch.org/get-started/locally/>.

It is highly recommended to use a virtual environment or a package manager such as Anaconda. Choose the GPU version if you have a NVIDIA GPU available on the computer.

PyTorch is already installed on all PCs in the computer rooms on campus, and it is possible to use them to work on the exercises. Also see <https://www.inf.uni-hamburg.de/en/inst/irz/infrastructure/pools.html> for information about which computer rooms on campus have a NVIDIA GPU on the computers. This can be very helpful later when you start working on the project and need a bit more computational power.

2.2 Basics of PyTorch

Tensors are multidimensional arrays similar to `ndarray` in NumPy. Tensors are used to represent data, such as images, or activation maps of a convolutional neural network.

Review and run the tutorial found at https://pytorch.org/tutorials/beginner/blitz/tensor_tutorial.html#getting-started.

3 PyTorch as an automatic differentiation engine

An important reason why PyTorch and other similar packages are used in machine learning is their capability for *automatic differentiation*. Consider the example function

$$f(x) = \cos(x), \quad (1)$$

with derivative

$$f'(x) = -\sin(x). \quad (2)$$

For example, we have $f'(\frac{\pi}{2}) = -1$. The following code snippet illustrates how automatic differentiation in PyTorch is used to get this result.

```
import torch
import math
x = torch.Tensor([math.pi / 2.0])
x.requires_grad_(True)
y = torch.cos(x)
print(y)
y.backward()
print(x.grad)
```

The code creates a tensor, and sets its `requires_grad` attribute to true to track operations that affect the derivative¹. The operation `torch.cos` saves its output to a new tensor `y`. When printing `y`, you will see an output similar to:

```
tensor([-4.3711e-08], grad_fn=<CosBackward>)
```

This indicates the tensor value (zero with some numerical inaccuracy), and its gradient function attribute. All operations have a `grad_fn` attribute that allows automatic differentiation. Calling `y.backward()` starts differentiation and accumulates derivatives to the `.grad` attribute of `x`.

Computation can be visualized as a directed acyclic graph as shown in Figure 1. This is known as a *computational graph*. Operations may take tensors as input, and output tensors. Connections are indicated by directed edges between the two types of nodes. Neural networks are complicated computational graphs with many layers of different types of operations.

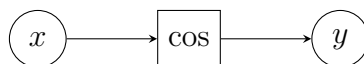


Figure 1: Computational graph of $y = \cos(x)$. The circle nodes are tensors, the rectangular node is an operation.

Further information about automatic differentiation can be found at https://pytorch.org/tutorials/beginner/blitz/autograd_tutorial.html. If you are interested in even more details, Elliot Waite has prepared a very nice technical overview of how automatic differentiation is implemented in PyTorch computational graphs in this video: <https://www.youtube.com/watch?v=MswxJw-8PvE>

Tasks.

1. Look at the computational graph in Figure 2. What is the function $z = f(x, y)$ represented by the graph? Write it down.

¹Note the underscore in `x.requires_grad_` signifying modifying the object in-place without copying.

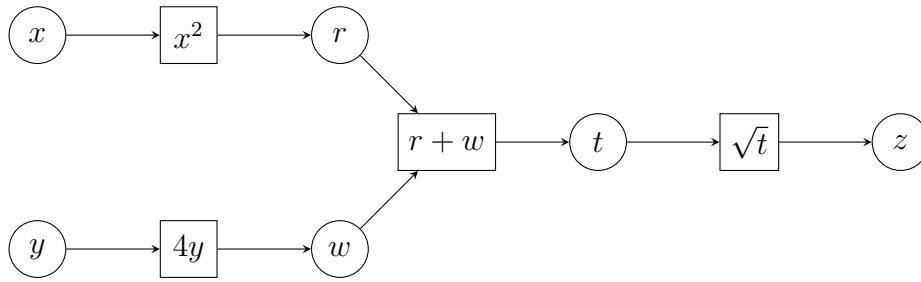


Figure 2: Computational graph of a function $z = f(x, y)$ of two variables.

2. Calculate the partial derivatives $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial y}$ on pen and paper. Hint: use the chain rule. The correct answers you should get are

$$\frac{\partial f}{\partial x} = \frac{x}{\sqrt{x^2 + 4y}} \quad (3)$$

and

$$\frac{\partial f}{\partial y} = \frac{2}{\sqrt{x^2 + 4y}}. \quad (4)$$

3. Calculate the values of the partial derivatives when $x = 4$ and $y = 4$.
4. Implement the computational graph in PyTorch, and use automatic differentiation to evaluate the partial derivatives.

Make sure to use floating point data types - you can check the datatype of a tensor $\mathbf{x}$ by inspecting or printing its property `$\mathbf{x}.\text{dtype}$` .

Neural networks and automatic differentiation. Neural networks are functions $y = f(x; \theta)$ that map an input x to an output y , and are parametrised by some vector θ of parameters. The parameters may consist of the weights and biases in a fully connected layers, or the kernel in a convolutional layer. The networks are often composed of n layers connected one after each other. Each layer is a function f_i with parameters θ_i . Then the overall network is the nested function $y = f_n(f_{n-1}(\dots f_1(x, \theta_1), \theta_2), \dots), \theta_n)$.

In PyTorch and similar packages, a layer of a neural network can be thought of as an operation f which has parameters θ that affect its output. We use a loss function L to measure how well the network is doing: high loss means poor performance, low loss means good performance. Now instead of computing the derivative with respect to the input, for each layer, we compute $\frac{\partial L}{\partial \theta}$ to find out how changing the parameters will affect the loss: will it increase or decrease? Then, the parameters can be changed to make the loss smaller.

Figure 3 shows an example of a computational graph of a neural network with two layers f_1 and f_2 . Part of the graph computes the nested function $y = f_2(f_1(x, \theta_1), \theta_2)$. In the next few exercises, we use automatic differentiation to update the parameters θ_i of the network to minimize the loss function L given some training dataset.

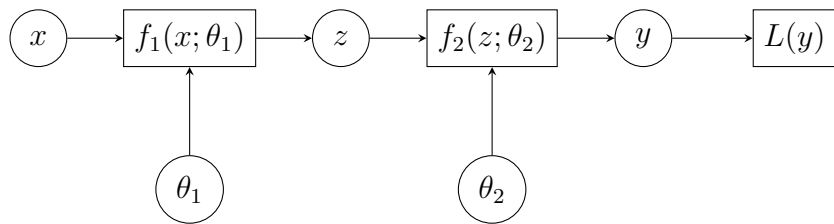


Figure 3: A two-layer neural network as a computational graph. Each layer f_i is parametrised by a vector θ_i of parameters. The loss function L can be differentiated with respect to θ_i by automatic differentiation to update the parameters.